

Capstone (part 1): Model Building for Autonomous car driving system : Classification and Regression of the location of the objects

1. Data Setup

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
# Importing of necessary libraries
import numpy as np
import pandas as pd

import tensorflow as tf

from keras.models import Sequential

import matplotlib.pyplot as plt
import shutil
import os
import cv2
import PIL as IMAGE
```

2. Image loading and verification

```
# Creating the path for the archive file and the extracted file for unzip
archive_file="/content/drive/MyDrive/Abhijit/Capstone/Images.zip"
extracted_path="/content/drive/MyDrive/Abhijit/Capstone"
```

```
# Unpacking the zip file with shutil library
shutil.unpack_archive(archive_file, extracted_path)
print(f"Successfully unpacked '{archive_file}' to '{extracted_path}'.")
```

```
# Load the data label file in data frame and check of the loaded data
```

```
df_labels=pd.read_csv(r"/content/drive/MyDrive/Abhijit/Capstone/labels.csv")
df_labels.head(5)
```

	id	car_type	x_min	y_min	x_max	y_max
0	0	pickup_truck	213	34	255	50
1	0	car	194	78	273	122
2	0	car	155	27	183	35
3	0	articulated_truck	43	25	109	55
4	0	car	106	32	124	45

```
# Labeling the column of the loaded data
df_labels.columns = ["IMAGE_ID", "CLASS", "X_MIN", "Y_MIN", "X_MAX", "Y_MAX"]
df_labels.head()
```

	IMAGE_ID	CLASS	X_MIN	Y_MIN	X_MAX	Y_MAX
0	0	pickup_truck	213	34	255	50
1	0	car	194	78	273	122
2	0	car	155	27	183	35
3	0	articulated_truck	43	25	109	55
4	0	car	106	32	124	45

```
#Slicing the data upto 500 rows and check the shape and head of the sliced data
df_subset = df_labels.iloc[:500]
print(df_subset.shape)
print(df_subset.head())
```

```
(500, 6)
  IMAGE_ID      CLASS  X_MIN  Y_MIN  X_MAX  Y_MAX
0         0  pickup_truck   213    34   255    50
1         0         car   194    78   273   122
2         0         car   155    27   183    35
3         0  articulated_truck    43    25   109    55
4         0         car   106    32   124    45
```

```
# Check of the some images present in the image directory
print(os.listdir("/content/drive/MyDrive/Abhijit/Capstone/Images")[:3])
```

```
['00000144.jpg', '00000145.jpg', '00000148.jpg']
```

```
# Check of the image path based on the image ID connected to the data file i.e. data_subset
extracted_path = "/content/drive/MyDrive/Abhijit/Capstone/Images"
```

```
image_id = df_subset.iloc[11]["IMAGE_ID"]
img_path = os.path.join(extracted_path, f"{image_id:08d}.jpg")
```

```
print(img_path)
```

```
/content/drive/MyDrive/Abhijit/Capstone/Images/00000002.jpg
```

```
# sanity check
import os
```

```
print(os.path.exists(img_path))
```

```
True
```

```
# read one images based on the image path
img = cv2.imread(img_path)
img
```

```
ndarray (480, 720, 3) show data
```



```
# check the number of files in the directory
len(os.listdir(extracted_path))
```

```
5626
```

3. Exploratory data Analysis

```
df_subset.columns
```

```
Index(['IMAGE_ID', 'CLASS', 'X_MIN', 'Y_MIN', 'X_MAX', 'Y_MAX'], dtype='object')
```

```
#check of the number of unique images present in the data
df_subset["IMAGE_ID"].unique()
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12,
        13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25,
        26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38,
        39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51,
        52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64,
        65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77,
        78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90,
        91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103,
        104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116,
        117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129,
        130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142,
        143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153])
```

```
# check of the classes of car and their counts
df_subset["CLASS"].value_counts()
```

	count
car	354
pickup_truck	50
motorized_vehicle	31
work_van	17
bus	14
articulated_truck	14
pedestrian	7
single_unit_truck	6
bicycle	5
non-motorized_vehicle	2

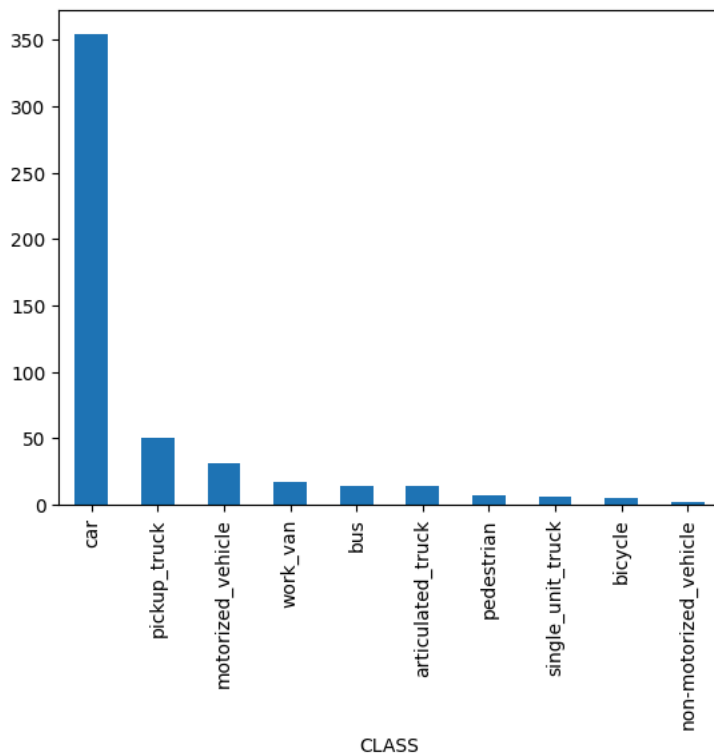
dtype: int64

```
len(df_subset["CLASS"])
```

500

```
df_subset["CLASS"].value_counts().plot(kind="bar") # To check the imbalance in the data
```

<Axes: xlabel='CLASS'>



```
# Checking the percentage of cars in 500 objects
per_car=len(df_subset[df_subset["CLASS"]=="car"])/len(df_subset)*100
per_car
```

70.8

```
# Grouping the ojects belongs to same image
for item in df_subset["CLASS"].unique():
    per_car=len(df_subset[df_subset["CLASS"]==item])/len(df_subset)*100
    print(f"{item}:{per_car:.2f}%")
```

```
pickup_truck:10.00%
car:70.80%
articulated_truck:2.80%
bus:2.80%
motorized_vehicle:6.20%
work_van:3.40%
single_unit_truck:1.20%
pedestrian:1.40%
bicycle:1.00%
non-motorized_vehicle:0.40%
```

Class imbalance:

1. Data for car class is huge ~71% Therefore highly imbalanced.
2. All other categories of images are also small. Like the percentages of bus is only 2.8% and pedestrian , non-motorized vehicle and bicycles classes are also small

```
df_subset["CLASS"].unique()
```

```
array(['pickup_truck', 'car', 'articulated_truck', 'bus',
      'motorized_vehicle', 'work_van', 'single_unit_truck', 'pedestrian',
      'bicycle', 'non-motorized_vehicle'], dtype=object)
```

```
# Encoding the class data
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
df_subset['label_encoded'] = le.fit_transform(df_subset['CLASS'])
```

/tmp/ipykernel_4044/2551047195.py:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df_subset['label_encoded'] = le.fit_transform(df_subset['CLASS'])

```
df_subset.head()
```

	IMAGE_ID	CLASS	X_MIN	Y_MIN	X_MAX	Y_MAX	label_encoded
0	0	pickup_truck	213	34	255	50	7
1	0	car	194	78	273	122	3
2	0	car	155	27	183	35	3
3	0	articulated_truck	43	25	109	55	0
4	0	car	106	32	124	45	3

Next steps: [Generate code with df_subset](#) [New interactive sheet](#)

```
grouped = df_subset.groupby("IMAGE_ID")
grouped
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x7fb5b9215c70>
```

```
grouped.head()
```

	IMAGE_ID	CLASS	X_MIN	Y_MIN	X_MAX	Y_MAX	label_encoded	
0	0	pickup_truck	213	34	255	50	7	
1	0	car	194	78	273	122	3	
2	0	car	155	27	183	35	3	
3	0	articulated_truck	43	25	109	55	0	
4	0	car	106	32	124	45	3	
...	...	...	...	...	...	...	...	
495	152	single_unit_truck	159	90	199	126	8	
496	152	car	512	257	688	403	3	
497	152	motorized_vehicle	158	92	205	117	4	
498	153	work_van	142	35	294	117	9	
499	153	car	332	33	478	104	3	

415 rows x 7 columns

len(grouped)

154

```
# Checking the group item
for img_id, group in grouped:
    print("Image ID:", img_id)
    print(group)
    break # just to see one group
```

```
Image ID: 0
IMAGE_ID    CLASS  X_MIN  Y_MIN  X_MAX  Y_MAX  label_encoded
0           0  pickup_truck  213    34   255    50           7
1           0         car    194    78   273   122           3
2           0         car    155    27   183    35           3
3           0  articulated_truck    43    25   109    55           0
4           0         car    106    32   124    45           3
```

df_subset.columns

```
Index(['IMAGE_ID', 'CLASS', 'X_MIN', 'Y_MIN', 'X_MAX', 'Y_MAX',
      'label_encoded'],
      dtype='object')
```

```
# Image list data creation based on the image-id and group and choose desire columns
image_data = []
```

```
for img_id, group in grouped:
    boxes = group[['X_MIN', 'Y_MIN', 'X_MAX', 'Y_MAX']].values
    labels = group['label_encoded'].values

    image_data.append({
        "image_id": img_id,
        "boxes": boxes,
        "labels": labels
    })
```

```
# check one image data
image_data[0]
```

```
{'image_id': 0,
 'boxes': array([[213,  34, 255,  50],
                [194,  78, 273, 122],
                [155,  27, 183,  35],
                [ 43,  25, 109,  55],
                [106,  32, 124,  45]]),
 'labels': array([7, 3, 3, 0, 3])}
```

```
# Preparation of the data set: Images, box corodinates, and image classes
import cv2
import numpy as np
```

```
X = []
y_bbox = []
y_class = []
```

IMG_SIZE = 224

```

for item in image_data:

    img_id = item["image_id"]

    # take first object (single detection)
    bbox = item["boxes"][0]
    label = item["labels"][0]

    # Corrected image path
    img_path = os.path.join(extracted_path, f"{img_id:08d}.jpg")

    img = cv2.imread(img_path)

    if img is None:
        print(f"Warning: Image not found or failed to load for ID {img_id} at path {img_path}. Skipping.")
        continue # Skip this item and proceed to the next one

    # original size
    h_orig, w_orig = img.shape[:2]

    # resize image
    img_resized = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
    img_resized = img_resized / 255.0

    # unpack bbox
    x_min, y_min, x_max, y_max = bbox

    # SCALE bbox to resized image

    scale_x = IMG_SIZE / w_orig
    scale_y = IMG_SIZE / h_orig

    x_min = x_min * scale_x
    x_max = x_max * scale_x
    y_min = y_min * scale_y
    y_max = y_max * scale_y

    # NORMALIZE (0-1)
    x_min /= IMG_SIZE
    x_max /= IMG_SIZE
    y_min /= IMG_SIZE
    y_max /= IMG_SIZE

    bbox_final = [x_min, y_min, x_max, y_max]

    # store
    X.append(img_resized)
    y_bbox.append(bbox_final)
    y_class.append(label)

# convert to numpy
X = np.array(X)
y_bbox = np.array(y_bbox)
y_class = np.array(y_class)

```

Warning: Image not found or failed to load for ID 92 at path /content/drive/MyDrive/Abhijit/Capstone/Images/00000

```
len(X)
```

```
153
```

```
len(y_bbox)
```

```
153
```

```
len(y_class)
```

```
153
```

```
len(y_class)
```

```
153
```

```

# Train and test data creation through the splitting
from sklearn.model_selection import train_test_split

X_train, X_test, y_bbox_train, y_bbox_test, y_class_train, y_class_test = train_test_split(
    X, y_bbox, y_class, test_size=0.2, random_state=42
)

```

```
# number of claases in the data
num_classes = len(df_subset["CLASS"].unique())
```

✓ A normal CNN model with multiple layers without pretrained VGG16 model

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Input
input_layer = tf.keras.Input(shape=(224, 224, 3))

# Convolutional Stack
x = layers.Conv2D(32, (3,3), activation='relu')(input_layer)
x = layers.MaxPooling2D((2,2))(x)
x = layers.BatchNormalization()(x)

x = layers.Conv2D(64, (3,3), activation='relu')(x)
x = layers.MaxPooling2D((2,2))(x)

x = layers.Conv2D(128, (3,3), activation='relu')(x)
x = layers.MaxPooling2D((2,2))(x)

# Dense / Shared Representation
x = layers.Flatten()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.3)(x)

# HEAD 1: Classification
class_output = layers.Dense(num_classes, activation='softmax', name='class_output')(x)

# HEAD 2: Bounding Box
bbox_output = layers.Dense(4, activation='sigmoid', name='bbox_output')(x)

# final Model
model = models.Model(inputs=input_layer, outputs=[class_output, bbox_output])

model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 3)	0	–
conv2d (Conv2D)	(None, 222, 222, 32)	896	input_layer[0][0]
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0	conv2d[0][0]
batch_normalization (BatchNormalizatio...	(None, 111, 111, 32)	128	max_pooling2d[0]...
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496	batch_normalizat...
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856	max_pooling2d_1[...
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0	conv2d_2[0][0]
flatten (Flatten)	(None, 86528)	0	max_pooling2d_2[...
dense (Dense)	(None, 128)	11,075,712	flatten[0][0]
dropout (Dropout)	(None, 128)	0	dense[0][0]
class_output (Dense)	(None, 10)	1,290	dropout[0][0]
bbox_output (Dense)	(None, 4)	516	dropout[0][0]

Total params: 11,170,894 (42.61 MB)
 Trainable params: 11,170,830 (42.61 MB)
 Non-trainable params: 64 (256.00 B)

```
# Compilation of the model
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
```

```

loss={
    'class_output': 'sparse_categorical_crossentropy',
    'bbox_output': 'mse' # but Huber loss is better than MSE
},
loss_weights={
    'class_output': 1.0,
    'bbox_output': 10.0 # IMPORTANT for bbox learning : weight high
},
metrics={
    'class_output': 'accuracy',
    'bbox_output': 'mae'
}
)

```

```

# Fitting data with the model
history = model.fit(
    X_train,
    {
        'class_output': y_class_train,
        'bbox_output': y_bbox_train
    },
    validation_data=(
        X_test,
        {
            'class_output': y_class_test,
            'bbox_output': y_bbox_test
        }
    ),
    batch_size=32,
    epochs=20
)

```

```

Epoch 1/20
4/4 ————— 24s 5s/step - bbox_output_loss: 0.1461 - bbox_output_mae: 0.3171 - class_output_accuracy
Epoch 2/20
4/4 ————— 21s 5s/step - bbox_output_loss: 0.1273 - bbox_output_mae: 0.2933 - class_output_accuracy
Epoch 3/20
4/4 ————— 16s 4s/step - bbox_output_loss: 0.0976 - bbox_output_mae: 0.2563 - class_output_accuracy
Epoch 4/20
4/4 ————— 22s 5s/step - bbox_output_loss: 0.0830 - bbox_output_mae: 0.2372 - class_output_accuracy
Epoch 5/20
4/4 ————— 15s 4s/step - bbox_output_loss: 0.0749 - bbox_output_mae: 0.2176 - class_output_accuracy
Epoch 6/20
4/4 ————— 20s 5s/step - bbox_output_loss: 0.0604 - bbox_output_mae: 0.1952 - class_output_accuracy
Epoch 7/20
4/4 ————— 21s 5s/step - bbox_output_loss: 0.0524 - bbox_output_mae: 0.1838 - class_output_accuracy
Epoch 8/20
4/4 ————— 16s 4s/step - bbox_output_loss: 0.0537 - bbox_output_mae: 0.1829 - class_output_accuracy
Epoch 9/20
4/4 ————— 19s 4s/step - bbox_output_loss: 0.0524 - bbox_output_mae: 0.1806 - class_output_accuracy
Epoch 10/20
4/4 ————— 20s 5s/step - bbox_output_loss: 0.0506 - bbox_output_mae: 0.1743 - class_output_accuracy
Epoch 11/20
4/4 ————— 15s 4s/step - bbox_output_loss: 0.0472 - bbox_output_mae: 0.1660 - class_output_accuracy
Epoch 12/20
4/4 ————— 24s 5s/step - bbox_output_loss: 0.0447 - bbox_output_mae: 0.1684 - class_output_accuracy
Epoch 13/20
4/4 ————— 21s 5s/step - bbox_output_loss: 0.0395 - bbox_output_mae: 0.1529 - class_output_accuracy
Epoch 14/20
4/4 ————— 15s 4s/step - bbox_output_loss: 0.0412 - bbox_output_mae: 0.1556 - class_output_accuracy
Epoch 15/20
4/4 ————— 19s 5s/step - bbox_output_loss: 0.0383 - bbox_output_mae: 0.1502 - class_output_accuracy
Epoch 16/20
4/4 ————— 16s 3s/step - bbox_output_loss: 0.0344 - bbox_output_mae: 0.1421 - class_output_accuracy
Epoch 17/20
4/4 ————— 20s 4s/step - bbox_output_loss: 0.0344 - bbox_output_mae: 0.1449 - class_output_accuracy
Epoch 18/20
4/4 ————— 20s 5s/step - bbox_output_loss: 0.0405 - bbox_output_mae: 0.1532 - class_output_accuracy
Epoch 19/20
4/4 ————— 20s 5s/step - bbox_output_loss: 0.0347 - bbox_output_mae: 0.1450 - class_output_accuracy
Epoch 20/20
4/4 ————— 17s 4s/step - bbox_output_loss: 0.0311 - bbox_output_mae: 0.1364 - class_output_accuracy

```

```

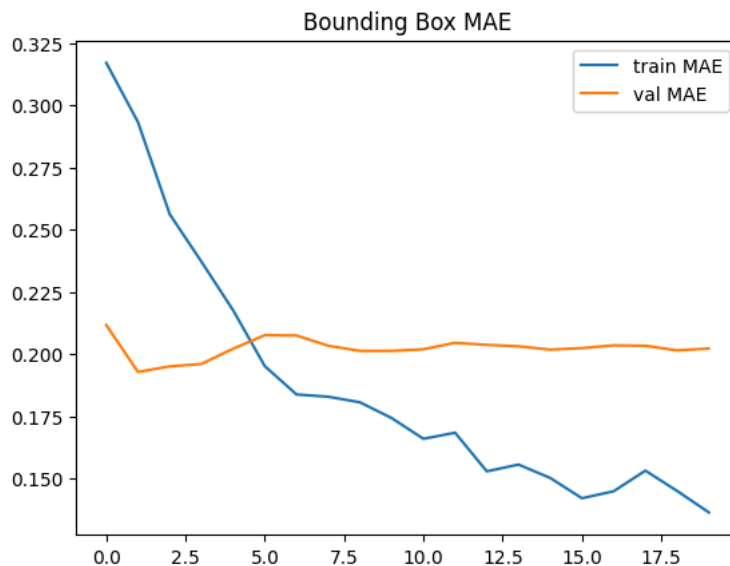
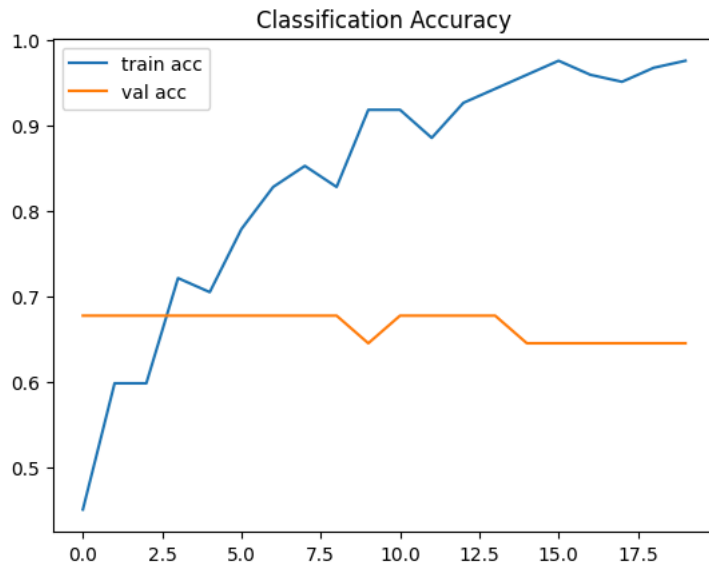
import matplotlib.pyplot as plt

# Classification accuracy
plt.plot(history.history['class_output_accuracy'], label='train acc')
plt.plot(history.history['val_class_output_accuracy'], label='val acc')
plt.legend()
plt.title("Classification Accuracy")
plt.show()

# Bounding box MAE

```

```
plt.plot(history.history['bbox_output_mae'], label='train MAE')
plt.plot(history.history['val_bbox_output_mae'], label='val MAE')
plt.legend()
plt.title("Bounding Box MAE")
plt.show()
```



```
# Evaluation of the model based on the training
results = model.evaluate(
    X_test,
    {
        'class_output': y_class_test,
        'bbox_output': y_bbox_test
    }
)

print(results)
```

```
1/1 ————— 1s 1s/step - bbox_output_loss: 0.0573 - bbox_output_mae: 0.2023 - class_output_accuracy:
[2.5197317600250244, 1.946913242340088, 0.057281844317913055, 0.20227032899856567, 0.6451612710952759]
```

```
# Prediction for an image
pred_class, pred_bbox = model.predict(X_test[:1])
print("Pred_class", pred_class)
print("Pred bbox:", pred_bbox[0])
print("True bbox:", y_bbox_test[0])
```

```
1/1 ————— 0s 286ms/step
Pred_class [[0.08644032 0.08552794 0.08844463 0.171136 0.09407657 0.08901442
0.08660927 0.09403519 0.10067347 0.10404221]]
Pred bbox: [0.4918134 0.4607981 0.52224714 0.47808412]
True bbox: [0.65789474 0.49561404 0.88888889 0.69736842]
```

```
# Conversion function of xyxy to the pixel dimension 224*224
def xyxy_to_pixel(box, img_size=224):
```

```
x_min, y_min, x_max, y_max = box

x_min = int(x_min * img_size)
y_min = int(y_min * img_size)
x_max = int(x_max * img_size)
y_max = int(y_max * img_size)

return x_min, y_min, x_max, y_max
```

```
# visualization Function for prediction output
import matplotlib.pyplot as plt
import cv2

def visualize_prediction(image, pred_box, true_box):

    img = (image * 255).astype("uint8").copy()

    # convert boxes
    pred_xyxy = xyxy_to_pixel(pred_box)
    true_xyxy = xyxy_to_pixel(true_box)

    # draw predicted box (GREEN)
    cv2.rectangle(img,
                  (pred_xyxy[0], pred_xyxy[1]),
                  (pred_xyxy[2], pred_xyxy[3]),
                  (0,255,0), 2)

    # draw true box (RED)
    cv2.rectangle(img,
                  (true_xyxy[0], true_xyxy[1]),
                  (true_xyxy[2], true_xyxy[3]),
                  (255,0,0), 2)

    plt.imshow(img)
    plt.title("Green = Predicted | Red = Ground Truth")
    plt.axis('off')
    plt.show()
```

```
# check prediction for an image data
idx = 0

image = X_test[idx]
true_box = y_bbox_test[idx] # Corrected: Removed the extra [0] index

pred_class, pred_bbox = model.predict(X_test[idx:idx+1])

pred_box = pred_bbox[0] # Select the first predicted bounding box for the first image

visualize_prediction(image, pred_box, true_box)
```

1/1 ————— 0s 58ms/step

Green = Predicted | Red = Ground Truth



Very bad prediction of the bounding box with the CNN model

```
# Building of the intersection over union loss function
def iou_loss(y_true, y_pred):
    x1 = tf.maximum(y_true[:,0], y_pred[:,0])
    y1 = tf.maximum(y_true[:,1], y_pred[:,1])
```

```

x2 = tf.minimum(y_true[:,2], y_pred[:,2])
y2 = tf.minimum(y_true[:,3], y_pred[:,3])

inter = tf.maximum(0.0, x2-x1) * tf.maximum(0.0, y2-y1)

area1 = (y_true[:,2]-y_true[:,0]) * (y_true[:,3]-y_true[:,1])
area2 = (y_pred[:,2]-y_pred[:,0]) * (y_pred[:,3]-y_pred[:,1])

union = area1 + area2 - inter

iou = inter / (union + 1e-6)

return 1 - iou

```

```

import tensorflow as tf

iou_threshold = 0.75
correct = 0

pred_class, pred_bbox = model.predict(X_test)

for i in range(len(X_test)):

    # predicted class
    pred_label = pred_class[i].argmax()

    # true class
    true_label = y_class_test[i]

    # predicted bbox
    # Expand dimensions to make it (1, 4) instead of (4,) and cast to float32
    pb_single = tf.cast(tf.expand_dims(pred_bbox[i], axis=0), tf.float32)

    # true bbox
    # Expand dimensions to make it (1, 4) instead of (4,) and cast to float32
    tb_single = tf.cast(tf.expand_dims(y_bbox_test[i], axis=0), tf.float32)

    # iou_loss returns (1 - actual_iou), so we subtract from 1 to get the actual IoU
    actual_iou = 1 - iou_loss(tb_single, pb_single)

    if pred_label == true_label and actual_iou >= iou_threshold:
        correct += 1

map_score = correct / len(X_test)

print("mAP (IoU=", iou_threshold, "):", map_score)

```

```

WARNING:tensorflow:5 out of the last 19 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_s
1/1 ----- 1s 1s/step
mAP (IoU= 0.75 ): 0.0

```

Very bad IoU score

✓ With Pretrained VGG16 Base model

```
from keras.applications.vgg16 import VGG16
```

```

from tensorflow.keras import layers, Model
import tensorflow as tf

def build_detection_model(num_classes):
    inputs = layers.Input(shape=(224, 224, 3))

    base_model = tf.keras.applications.VGG16(
        include_top=False, weights='imagenet', input_tensor=inputs
    )
    base_model.trainable = False

    x = base_model.output

    x = layers.Conv2D(128, (3,3), activation='relu', padding='same')(x)
    x = layers.BatchNormalization()(x)

    #
    x = layers.GlobalAveragePooling2D()(x)

    x = layers.Dense(128, activation='relu')(x)

```

```
x = layers.Dropout(0.4)(x)

class_out = layers.Dense(num_classes, activation='softmax', name='class_output')(x)
bbox_out = layers.Dense(4, activation='sigmoid', name='bbox_output')(x)

model = Model(inputs=inputs, outputs=[class_out, bbox_out])

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-4),
    loss={
        'class_output': 'sparse_categorical_crossentropy',
        'bbox_output': 'mse'
    },
    loss_weights={'class_output': 1.0, 'bbox_output': 5.0},
    metrics={'class_output': 'accuracy', 'bbox_output': 'mse'}
)

return model

model_VGG = build_detection_model(num_classes)
```

```
model_VGG.summary()
```

Model: "functional_5"

Layer (type)	Output Shape	Param #	Connected to
input_layer_5 (InputLayer)	(None, 224, 224, 3)	0	–
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792	input_layer_5[0]...
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928	block1_conv1[0] [...
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0	block1_conv2[0] [...
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856	block1_pool[0][0]
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584	block2_conv1[0] [...
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0	block2_conv2[0] [...
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168	block2_pool[0][0]
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080	block3_conv1[0] [...
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080	block3_conv2[0] [...
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0	block3_conv3[0] [...
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160	block3_pool[0][0]
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808	block4_conv1[0] [...
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808	block4_conv2[0] [...
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0	block4_conv3[0] [...
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808	block4_pool[0][0]
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808	block5_conv1[0] [...
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808	block5_conv2[0] [...
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0	block5_conv3[0] [...
conv2d_7 (Conv2D)	(None, 7, 7, 128)	589,952	block5_pool[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 7, 7, 128)	512	conv2d_7[0][0]
global_average_poo... (GlobalAveragePool...	(None, 128)	0	batch_normalizat...
dense_5 (Dense)	(None, 128)	16,512	global_average_p...
dropout_5 (Dropout)	(None, 128)	0	dense_5[0][0]
class_output (Dense)	(None, 10)	1,290	dropout_5[0][0]
bbox_output (Dense)	(None, 4)	516	dropout_5[0][0]

Total params: 15,323,470 (58.45 MB)
Trainable params: 608,526 (2.32 MB)
Non-trainable params: 14,714,944 (56.13 MB)

```

# Fitting data with the VGG16 model
history = model_VGG.fit(
    X_train,
    {
        'class_output': y_class_train,
        'bbox_output': y_bbox_train
    },
    validation_data=(
        X_test,
        {

```

```

        'class_output': y_class_test,
        'bbox_output': y_bbox_test
    }
),
batch_size=32,
epochs=20
)

```

```

Epoch 1/20
4/4 ————— 84s 22s/step - bbox_output_loss: 0.0482 - bbox_output_mse: 0.0478 - class_output_accurac
Epoch 2/20
4/4 ————— 141s 22s/step - bbox_output_loss: 0.0497 - bbox_output_mse: 0.0497 - class_output_accurac
Epoch 3/20
4/4 ————— 86s 23s/step - bbox_output_loss: 0.0473 - bbox_output_mse: 0.0467 - class_output_accurac
Epoch 4/20
4/4 ————— 138s 22s/step - bbox_output_loss: 0.0538 - bbox_output_mse: 0.0535 - class_output_accurac
Epoch 5/20
4/4 ————— 86s 23s/step - bbox_output_loss: 0.0401 - bbox_output_mse: 0.0403 - class_output_accurac
Epoch 6/20
4/4 ————— 86s 23s/step - bbox_output_loss: 0.0471 - bbox_output_mse: 0.0468 - class_output_accurac
Epoch 7/20
4/4 ————— 139s 22s/step - bbox_output_loss: 0.0421 - bbox_output_mse: 0.0421 - class_output_accurac
Epoch 8/20
4/4 ————— 86s 23s/step - bbox_output_loss: 0.0407 - bbox_output_mse: 0.0406 - class_output_accurac
Epoch 9/20
4/4 ————— 142s 23s/step - bbox_output_loss: 0.0397 - bbox_output_mse: 0.0395 - class_output_accurac
Epoch 10/20
4/4 ————— 142s 23s/step - bbox_output_loss: 0.0427 - bbox_output_mse: 0.0423 - class_output_accurac
Epoch 11/20
4/4 ————— 88s 24s/step - bbox_output_loss: 0.0470 - bbox_output_mse: 0.0470 - class_output_accurac
Epoch 12/20
4/4 ————— 87s 23s/step - bbox_output_loss: 0.0437 - bbox_output_mse: 0.0436 - class_output_accurac
Epoch 13/20
4/4 ————— 142s 23s/step - bbox_output_loss: 0.0430 - bbox_output_mse: 0.0430 - class_output_accurac
Epoch 14/20
4/4 ————— 142s 23s/step - bbox_output_loss: 0.0383 - bbox_output_mse: 0.0386 - class_output_accurac
Epoch 15/20
4/4 ————— 138s 22s/step - bbox_output_loss: 0.0407 - bbox_output_mse: 0.0402 - class_output_accurac
Epoch 16/20
4/4 ————— 83s 22s/step - bbox_output_loss: 0.0395 - bbox_output_mse: 0.0393 - class_output_accurac
Epoch 17/20
4/4 ————— 88s 23s/step - bbox_output_loss: 0.0365 - bbox_output_mse: 0.0365 - class_output_accurac
Epoch 18/20
4/4 ————— 87s 23s/step - bbox_output_loss: 0.0404 - bbox_output_mse: 0.0401 - class_output_accurac
Epoch 19/20
4/4 ————— 142s 23s/step - bbox_output_loss: 0.0434 - bbox_output_mse: 0.0436 - class_output_accurac
Epoch 20/20
4/4 ————— 89s 24s/step - bbox_output_loss: 0.0356 - bbox_output_mse: 0.0355 - class_output_accurac

```

```

# Evaluation of the model based on the training
results_VGG = model_VGG.evaluate(
    X_test,
    {
        'class_output': y_class_test,
        'bbox_output': y_bbox_test
    }
)

print(results_VGG)

```

```

1/1 ————— 18s 18s/step - bbox_output_loss: 0.0505 - bbox_output_mse: 0.0505 - class_output_accurac
[1.6665623188018799, 1.4139647483825684, 0.050519514828920364, 0.050519514828920364, 0.5806451439857483]

```

```

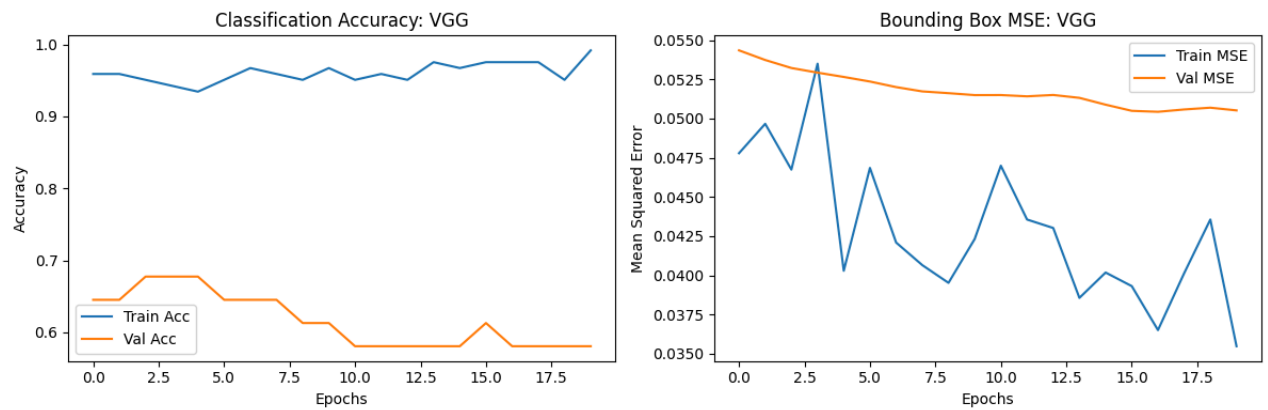
import matplotlib.pyplot as plt

# 1. Classification Accuracy Plot
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['class_output_accuracy'], label='Train Acc')
plt.plot(history.history['val_class_output_accuracy'], label='Val Acc')
plt.title("Classification Accuracy: VGG")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()

# 2. Bounding Box MSE Plot
plt.subplot(1, 2, 2)
plt.plot(history.history['bbox_output_mse'], label='Train MSE')
plt.plot(history.history['val_bbox_output_mse'], label='Val MSE')
plt.title("Bounding Box MSE: VGG")
plt.xlabel("Epochs")
plt.ylabel("Mean Squared Error")
plt.legend()

```

```
plt.tight_layout()
plt.show()
```



- Model accuracy for training increases but validation accuracy not changing at all. Possibly less number of train data be the reason
- Bounding box mse decreases for the training data more effective over the validation data - Model is not capturing the coordinates properly for the validation set maybe the model is overfitted

```
# Prediction class and coordinate for an image belongs to X_test data
pred_class_VGG, pred_bbox_VGG = model_VGG.predict(X_test[:1])
print("Pred_class", pred_class_VGG)
print("Pred bbox:", pred_bbox_VGG[0])
print("True bbox:", y_bbox_test[0])
```

```
1/1 ————— 1s 748ms/step
Pred_class [[0.07575166 0.07303668 0.06570903 0.35557353 0.0578484 0.06164909
 0.07424619 0.09063835 0.07507913 0.07046804]]
Pred bbox: [0.4795222 0.46265507 0.55571973 0.49351504]
True bbox: [0.65789474 0.49561404 0.88888889 0.69736842]
```

```
# Conversion function of xyxy to the pixel dimension 224*224
def xyxy_to_pixel(box, img_size=224):
    x_min, y_min, x_max, y_max = box

    x_min = int(x_min * img_size)
    y_min = int(y_min * img_size)
    x_max = int(x_max * img_size)
    y_max = int(y_max * img_size)

    return x_min, y_min, x_max, y_max
```

```
# visualization Function for prediction output
import matplotlib.pyplot as plt
import cv2

def visualize_prediction(image, pred_box, true_box):

    img = (image * 255).astype("uint8").copy()

    # convert boxes
    pred_xyxy = xyxy_to_pixel(pred_box)
    true_xyxy = xyxy_to_pixel(true_box)

    # draw predicted box (GREEN)
    cv2.rectangle(img,
                  (pred_xyxy[0], pred_xyxy[1]),
                  (pred_xyxy[2], pred_xyxy[3]),
                  (0,255,0), 2)

    # draw true box (RED)
    cv2.rectangle(img,
                  (true_xyxy[0], true_xyxy[1]),
                  (true_xyxy[2], true_xyxy[3]),
                  (255,0,0), 2)

    plt.imshow(img)
    plt.title("Green = Predicted | Red = Ground Truth")
    plt.axis('off')
    plt.show()
```

```
# check prediction for an image data
idx = 0

image = X_test[idx]
true_box = y_bbox_test[idx] # Corrected: Removed the extra [0] index

pred_class_VGG, pred_bbox_VGG = model_VGG.predict(X_test[idx:idx+1])

pred_box_VGG = pred_bbox_VGG[0] # Select the first predicted bounding box for the first image

visualize_prediction(image, pred_box_VGG, true_box)
```

1/1 ————— 1s 513ms/step

Green = Predicted | Red = Ground Truth



Very bad prediction of the bounding box even after using the VGG16 base models with pretrained weights

```
# Building of the intersection over union loss function
def iou_loss(y_true, y_pred):
    x1 = tf.maximum(y_true[:,0], y_pred[:,0])
    y1 = tf.maximum(y_true[:,1], y_pred[:,1])
    x2 = tf.minimum(y_true[:,2], y_pred[:,2])
    y2 = tf.minimum(y_true[:,3], y_pred[:,3])

    inter = tf.maximum(0.0, x2-x1) * tf.maximum(0.0, y2-y1)

    area1 = (y_true[:,2]-y_true[:,0]) * (y_true[:,3]-y_true[:,1])
    area2 = (y_pred[:,2]-y_pred[:,0]) * (y_pred[:,3]-y_pred[:,1])

    union = area1 + area2 - inter
```

```
iou = inter / (union + 1e-6)

return 1 - iou
```

```
import tensorflow as tf

iou_threshold = 0.75
correct = 0

pred_class_VGG, pred_bbox_VGG = model_VGG.predict(X_test)

for i in range(len(X_test)):

    # predicted class
    pred_label = pred_class_VGG[i].argmax()

    # true class
    true_label = y_class_test[i]

    # predicted bbox
    # Expand dimensions to make it (1, 4) instead of (4,) and cast to float32
    pb_single = tf.cast(tf.expand_dims(pred_bbox_VGG[i], axis=0), tf.float32)

    # true bbox
    # Expand dimensions to make it (1, 4) instead of (4,) and cast to float32
    tb_single = tf.cast(tf.expand_dims(y_bbox_test[i], axis=0), tf.float32)

    # iou_loss returns (1 - actual_iou), so we subtract from 1 to get the actual IoU
    actual_iou = 1 - iou_loss(tb_single, pb_single)

    if pred_label == true_label and actual_iou >= iou_threshold:
        correct += 1

map_score = correct / len(X_test)

print("mAP (IoU=", iou_threshold, "):", map_score)
```

1/1  17s 17s/step
mAP (IoU= 0.75): 0.0

```
pred_class_VGG, pred_bbox_VGG = model_VGG.predict(X_test[:5])

print("Pred bbox:", pred_bbox_VGG[0])
print("True bbox:", y_bbox_test[0])
```

1/1  4s 4s/step
Pred bbox: [0.4795222 0.46265507 0.5557196 0.49351507]
True bbox: [0.65789474 0.49561404 0.88888889 0.69736842]

✓ Augmentation of the data and model building with the pretrained VGG model

VGG model with Augmented data

```
# image Augmentation library
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
advanced_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=10,
    width_shift_range=0.05,
    height_shift_range=0.05,
    zoom_range=0.05,
    brightness_range=[0.8, 1.2]
)
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

def augmented_generator(X, y_class, y_bbox, batch_size):
    # Using the updated 'advanced_datagen' here
    gen = advanced_datagen.flow(X, batch_size=batch_size, shuffle=True)

    while True:
        batch_X = next(gen)

        # Get indices of this batch to align labels and bounding boxes
```

```

idx = gen.index_array[:len(batch_X)]

batch_y_class = y_class[idx]
batch_y_bbox = y_bbox[idx]

yield batch_X, {
    'class_output': batch_y_class,
    'bbox_output': batch_y_bbox
}

```

#VGG Model fit with the Augmented data with decreasing the batch size and with augmented data
batch_size = 16

```

train_gen = augmented_generator(X_train, y_class_train, y_bbox_train, batch_size)

history_VGG_Aug=model_VGG.fit(
    train_gen,
    steps_per_epoch=len(X_train)//batch_size,
    validation_data=(
        X_test,
        {
            'class_output': y_class_test,
            'bbox_output': y_bbox_test
        }
    ),
    epochs=20
)

```

```

bbox_output_mse: 0.0710 - class_output_accuracy: 0.6250 - class_output_loss: 2.1237 - loss: 2.4789 - val_bbox_outp
bbox_output_mse: 0.0842 - class_output_accuracy: 0.6887 - class_output_loss: 2.0953 - loss: 2.5165 - val_bbox_outp
bbox_output_mse: 0.0712 - class_output_accuracy: 0.6509 - class_output_loss: 2.0836 - loss: 2.4413 - val_bbox_outp
bbox_output_mse: 0.0727 - class_output_accuracy: 0.7170 - class_output_loss: 2.0451 - loss: 2.4098 - val_bbox_outp
bbox_output_mse: 0.0731 - class_output_accuracy: 0.6604 - class_output_loss: 2.0447 - loss: 2.4109 - val_bbox_outp
bbox_output_mse: 0.0652 - class_output_accuracy: 0.5189 - class_output_loss: 2.0752 - loss: 2.4018 - val_bbox_outp
bbox_output_mse: 0.0582 - class_output_accuracy: 0.4811 - class_output_loss: 2.0765 - loss: 2.3635 - val_bbox_outp
bbox_output_mse: 0.0619 - class_output_accuracy: 0.8113 - class_output_loss: 1.8867 - loss: 2.1966 - val_bbox_outp
bbox_output_mse: 0.0687 - class_output_accuracy: 0.5625 - class_output_loss: 2.0007 - loss: 2.3440 - val_bbox_outp
bbox_output_mse: 0.0697 - class_output_accuracy: 0.6698 - class_output_loss: 1.9223 - loss: 2.2646 - val_bbox_outp
bbox_output_mse: 0.0706 - class_output_accuracy: 0.7453 - class_output_loss: 1.8543 - loss: 2.2061 - val_bbox_outp
bbox_output_mse: 0.0783 - class_output_accuracy: 0.6415 - class_output_loss: 1.8881 - loss: 2.2869 - val_bbox_outp
bbox_output_mse: 0.0706 - class_output_accuracy: 0.5472 - class_output_loss: 1.9486 - loss: 2.2939 - val_bbox_outp
bbox_output_mse: 0.0619 - class_output_accuracy: 0.6132 - class_output_loss: 1.8615 - loss: 2.1738 - val_bbox_outp
bbox_output_mse: 0.0911 - class_output_accuracy: 0.6509 - class_output_loss: 1.8283 - loss: 2.2898 - val_bbox_outp
bbox_output_mse: 0.0822 - class_output_accuracy: 0.4151 - class_output_loss: 2.0398 - loss: 2.4386 - val_bbox_outp
bbox_output_mse: 0.0590 - class_output_accuracy: 0.8125 - class_output_loss: 1.6131 - loss: 1.9079 - val_bbox_outp
bbox_output_mse: 0.0665 - class_output_accuracy: 0.6509 - class_output_loss: 1.7285 - loss: 2.0689 - val_bbox_outp
bbox_output_mse: 0.0597 - class_output_accuracy: 0.5755 - class_output_loss: 1.7966 - loss: 2.0951 - val_bbox_outp
bbox_output_mse: 0.0526 - class_output_accuracy: 0.6698 - class_output_loss: 1.6840 - loss: 1.9365 - val_bbox_outp

```

```

# Evaluation of the model based on the training
results_VGG_AUG = model_VGG.evaluate(
    X_test,
    {
        'class_output': y_class_test,
        'bbox_output': y_bbox_test
    }
)

print(results_VGG_AUG)

```

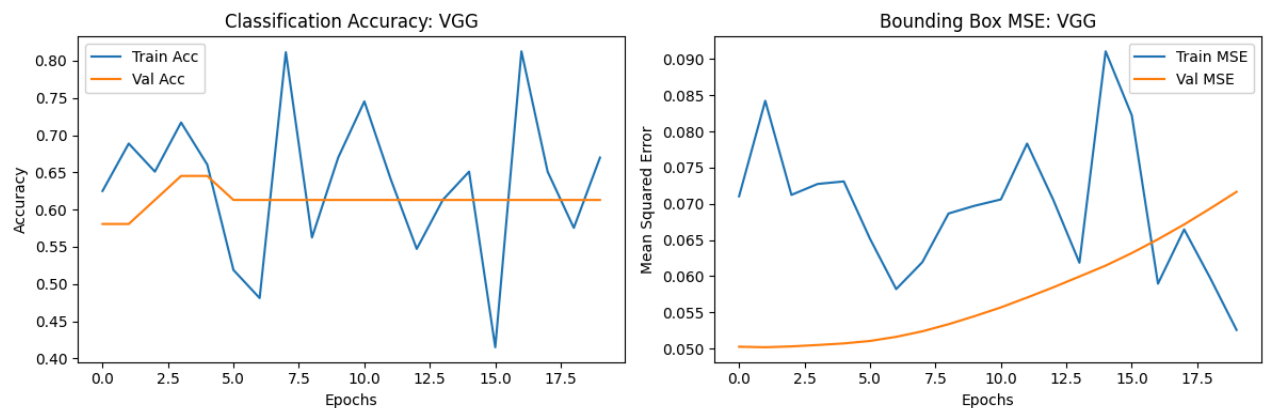
1/1 ————— 17s 17s/step - bbox_output_loss: 0.0717 - bbox_output_mse: 0.0717 - class_output_accu
[1.966860294342041, 1.6085691452026367, 0.07165822386741638, 0.07165822386741638, 0.6129032373428345]

```
import matplotlib.pyplot as plt

# 1. Classification Accuracy Plot
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history_VGG_Aug.history['class_output_accuracy'], label='Train Acc')
plt.plot(history_VGG_Aug.history['val_class_output_accuracy'], label='Val Acc')
plt.title("Classification Accuracy: VGG")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()

# 2. Bounding Box MSE Plot
plt.subplot(1, 2, 2)
plt.plot(history_VGG_Aug.history['bbox_output_mse'], label='Train MSE')
plt.plot(history_VGG_Aug.history['val_bbox_output_mse'], label='Val MSE')
plt.title("Bounding Box MSE: VGG")
plt.xlabel("Epochs")
plt.ylabel("Mean Squared Error")
plt.legend()

plt.tight_layout()
plt.show()
```



Accuracy is not improving so much with the augmented data POSSibly model parameters from the base VGG16 model is freeze. If we train last few layers of the base model the accuracy may improve

```
# Prediction for an image
pred_class_VGG_AUG, pred_bbox_VGG_AUG = model_VGG.predict(X_test[:1])
print("Pred_class", pred_class_VGG_AUG)
print("Pred bbox:", pred_bbox_VGG_AUG[0])
print("True bbox:", y_bbox_test[0])
```

```
1/1 ————— 1s 847ms/step
Pred_class [[0.03047061 0.03481393 0.04677362 0.6067858 0.00883674 0.04124359
0.10340068 0.07043283 0.02433193 0.03291025]]
Pred bbox: [0.36818752 0.22493042 0.6414073 0.38807008]
True bbox: [0.65789474 0.49561404 0.88888889 0.69736842]
```

```
df_class=df_subset[['CLASS', 'label_encoded']].drop_duplicates().sort_values('label_encoded')
df_class['CLASS'].to_list()
```

```
['articulated_truck',
'bicycle',
'bus',
'car',
'motorized_vehicle',
'non-motorized_vehicle',
'pedestrian',
'pickup_truck',
'single_unit_truck',
'work_van']
```

```
import tensorflow as tf
import numpy as np
import requests
from PIL import Image
```

```

from io import BytesIO
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def test_model_on_internet_image(img_url, model, class_names, img_size=224):
    # 1. Download the image
    response = requests.get(img_url)
    img = Image.open(BytesIO(response.content)).convert('RGB')

    # 2. Preprocess (Must match your Training exactly!)
    img_resized = img.resize((img_size, img_size))
    img_array = np.array(img_resized) / 255.0 # Rescale 1./255
    img_array = np.expand_dims(img_array, axis=0) # Add batch dimension

    # 3. Predict
    predictions = model.predict(img_array)

    # Extract results based on your model's output names
    # Assuming predictions is a list: [class_output_array, bbox_output_array]
    pred_class_logits = predictions[0] # First element is class output
    pred_bbox = predictions[1][0]     # Second element is bbox output

    # Get class name
    class_idx = np.argmax(pred_class_logits)
    predicted_label = class_names[class_idx]
    confidence = np.max(tf.nn.softmax(pred_class_logits))

    # 4. Visualize
    fig, ax = plt.subplots(1)
    ax.imshow(img_resized)

    # Draw Bounding Box
    # Convert normalized [0,1] to pixel values
    xmin, ymin, xmax, ymax = pred_bbox
    width = (xmax - xmin) * img_size
    height = (ymax - ymin) * img_size

    rect = patches.Rectangle((xmin * img_size, ymin * img_size),
                             width, height,
                             linewidth=2, edgecolor='r', facecolor='none')

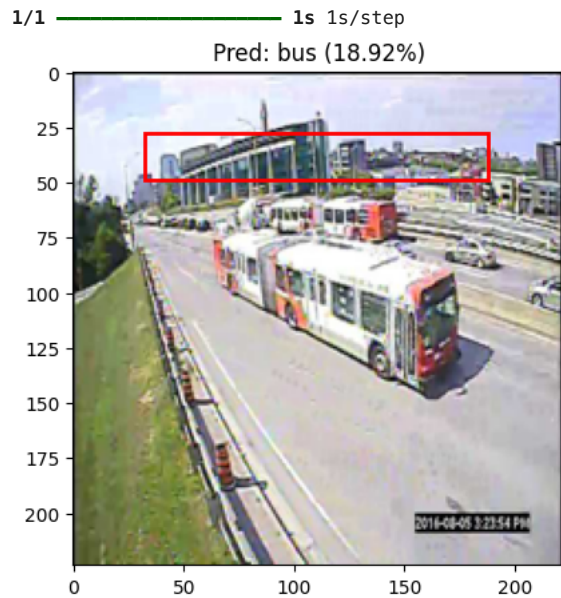
    ax.add_patch(rect)

    plt.title(f"Pred: {predicted_label} ({confidence*100:.2f}%)")
    plt.show()

# --- Execution ---
# Replace with your actual list of classes
my_classes = ['articulated_truck',
              'bicycle',
              'bus',
              'car',
              'motorized_vehicle',
              'non-motorized_vehicle',
              'pedestrian',
              'pickup_truck',
              'single_unit_truck',
              'work_van']

# Example URL
# url = "https://drive.google.com/file/d/14lB-RP8Ip6Foclrl-S6EFhp0yntjks07/view?usp=sharing"
url = "https://drive.google.com/uc?export=download&id=14lB-RP8Ip6Foclrl-S6EFhp0yntjks07"
test_model_on_internet_image(url, model_VGG, my_classes)

```



```
# Conversion function of xyxy to the pixel dimension 224*224
def xyxy_to_pixel(box, img_size=224):
    x_min, y_min, x_max, y_max = box

    x_min = int(x_min * img_size)
    y_min = int(y_min * img_size)
    x_max = int(x_max * img_size)
    y_max = int(y_max * img_size)

    return x_min, y_min, x_max, y_max
```

```
# visualization Function for prediction output
import matplotlib.pyplot as plt
import cv2

def visualize_prediction(image, pred_box, true_box):

    img = (image * 255).astype("uint8").copy()

    # convert boxes
    pred_xyxy = xyxy_to_pixel(pred_box)
    true_xyxy = xyxy_to_pixel(true_box)

    # draw predicted box (GREEN)
    cv2.rectangle(img,
                  (pred_xyxy[0], pred_xyxy[1]),
                  (pred_xyxy[2], pred_xyxy[3]),
                  (0,255,0), 2)

    # draw true box (RED)
    cv2.rectangle(img,
                  (true_xyxy[0], true_xyxy[1]),
                  (true_xyxy[2], true_xyxy[3]),
                  (255,0,0), 2)

    plt.imshow(img)
    plt.title("Green = Predicted | Red = Ground Truth")
    plt.axis('off')
    plt.show()
```

```
# check prediction for an image data
idx = 1

image = X_test[idx]
true_box = y_bbox_test[idx] # Corrected: Removed the extra [0] index

pred_class_VGG_AUG, pred_bbox_VGG_AUG = model_VGG.predict(X_test[idx:idx+1])

pred_box_VGG_AUG = pred_bbox_VGG_AUG[0] # Select the first predicted bounding box for the first image

visualize_prediction(image, pred_box_VGG_AUG, true_box)
```

1/1 ————— 1s 976ms/step

Green = Predicted | Red = Ground Truth



Still it is not a good prediction of the bounding box even after Augmentaion of the train data

```
# Building of the intersection over union loss function
def iou_loss(y_true, y_pred):
    x1 = tf.maximum(y_true[:,0], y_pred[:,0])
    y1 = tf.maximum(y_true[:,1], y_pred[:,1])
    x2 = tf.minimum(y_true[:,2], y_pred[:,2])
    y2 = tf.minimum(y_true[:,3], y_pred[:,3])

    inter = tf.maximum(0.0, x2-x1) * tf.maximum(0.0, y2-y1)

    area1 = (y_true[:,2]-y_true[:,0]) * (y_true[:,3]-y_true[:,1])
    area2 = (y_pred[:,2]-y_pred[:,0]) * (y_pred[:,3]-y_pred[:,1])

    union = area1 + area2 - inter

    iou = inter / (union + 1e-6)

    return 1 - iou
```

```
import tensorflow as tf

iou_threshold = 0.75
correct = 0

pred_class_VGG_AUG, pred_bbox_VGG_AUG = model_VGG.predict(X_test)

for i in range(len(X_test)):

    # predicted class
    pred_label = pred_class_VGG_AUG[i].argmax()

    # true class
    true_label = y_class_test[i]

    # predicted bbox
    # Expand dimensions to make it (1, 4) instead of (4,) and cast to float32
    pb_single = tf.cast(tf.expand_dims(pred_bbox_VGG_AUG[i], axis=0), tf.float32)

    # true bbox
    # Expand dimensions to make it (1, 4) instead of (4,) and cast to float32
    tb_single = tf.cast(tf.expand_dims(y_bbox_test[i], axis=0), tf.float32)

    # iou_loss returns (1 - actual_iou), so we subtract from 1 to get the actual IoU
    actual_iou = 1 - iou_loss(tb_single, pb_single)

    if pred_label == true_label and actual_iou >= iou_threshold:
        correct += 1

map_score = correct / len(X_test)

print("mAP (IoU=", iou_threshold, "):", map_score)
```

1/1 ————— 17s 17s/step

mAP (IoU= 0.75): 0.0

```
pred_class_VGG_AUG, pred_bbox_VGG_AUG = model_VGG.predict(X_test[:5])
```

```
print("Pred bbox:", pred_bbox_VGG_AUG[0])
print("True bbox:", y_bbox_test[0])
```

```
1/1 ————— 4s 4s/step
Pred bbox: [0.36818752 0.22493042 0.6414073 0.3880701 ]
True bbox: [0.65789474 0.49561404 0.88888889 0.69736842]
```

1. Accuracy and bounding box prediction are become worst here as the model uses the augmented data.
2. It is unclear the origin of the problem in the code. Now it is predicting the box with valid coordinates but the true and predicted box are far apart
3. MAP IoU score is also less

✓ With trainable layers of VGG16 as base model with the Augmented data

```
# 1. UNFREEZE: Open the last 10 layers for training
for layer in model_VGG.layers:
    if layer.name == 'vgg16':
        layer.trainable = True
        for sub_layer in layer.layers[:-10]:
            sub_layer.trainable = False
    else:
        layer.trainable = True # Keep your custom heads trainable

# 2. Compilation
model_VGG.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss={
        'class_output': 'sparse_categorical_crossentropy',
        'bbox_output': 'mse' # Huber is more stable for fine-tuning boxes
    },
    loss_weights={
        'class_output': 1.0,
        'bbox_output': 5.0
    },
    metrics={
        'class_output': 'accuracy',
        'bbox_output': 'mae'
    }
)

# 3. FIT: Train on the augmented data
batch_size = 16
EPOCHS = 20

history_VGG_Aug_LT = model_VGG.fit(
    train_gen, # augmented generator
    steps_per_epoch=len(X_train) // batch_size,
    validation_data=(
        X_test,
        {
            'class_output': y_class_test,
            'bbox_output': y_bbox_test
        }
    ),
    epochs=EPOCHS
)
```

```
:put_mae: 0.2227 - class_output_accuracy: 0.6875 - class_output_loss: 1.6187 - loss: 1.9835 - val_bbox_output_loss
:put_mae: 0.2218 - class_output_accuracy: 0.6698 - class_output_loss: 1.6692 - loss: 2.0087 - val_bbox_output_loss
:put_mae: 0.2090 - class_output_accuracy: 0.7830 - class_output_loss: 1.5480 - loss: 1.8462 - val_bbox_output_loss
:put_mae: 0.2080 - class_output_accuracy: 0.7830 - class_output_loss: 1.5355 - loss: 1.8429 - val_bbox_output_loss
:put_mae: 0.2149 - class_output_accuracy: 0.6887 - class_output_loss: 1.6208 - loss: 1.9377 - val_bbox_output_loss
:put_mae: 0.2168 - class_output_accuracy: 0.6792 - class_output_loss: 1.6506 - loss: 1.9600 - val_bbox_output_loss
:put_mae: 0.2167 - class_output_accuracy: 0.8019 - class_output_loss: 1.5038 - loss: 1.8195 - val_bbox_output_loss
:put_mae: 0.2008 - class_output_accuracy: 0.6887 - class_output_loss: 1.6124 - loss: 1.8944 - val_bbox_output_loss
:put_mae: 0.2112 - class_output_accuracy: 0.6250 - class_output_loss: 1.7234 - loss: 2.0151 - val_bbox_output_loss
:put_mae: 0.2434 - class_output_accuracy: 0.7453 - class_output_loss: 1.5812 - loss: 1.9660 - val_bbox_output_loss
```

```
:put_mae: 0.2307 - class_output_accuracy: 0.6981 - class_output_loss: 1.5932 - loss: 1.9548 - val_bbox_output_loss
:put_mae: 0.2218 - class_output_accuracy: 0.7170 - class_output_loss: 1.5542 - loss: 1.8710 - val_bbox_output_loss
:put_mae: 0.2201 - class_output_accuracy: 0.7547 - class_output_loss: 1.5624 - loss: 1.8780 - val_bbox_output_loss
:put_mae: 0.2236 - class_output_accuracy: 0.7547 - class_output_loss: 1.5370 - loss: 1.8561 - val_bbox_output_loss
:put_mae: 0.2223 - class_output_accuracy: 0.7830 - class_output_loss: 1.4882 - loss: 1.8195 - val_bbox_output_loss
:put_mae: 0.2433 - class_output_accuracy: 0.6226 - class_output_loss: 1.7125 - loss: 2.1004 - val_bbox_output_loss
:put_mae: 0.1778 - class_output_accuracy: 0.6875 - class_output_loss: 1.5887 - loss: 1.8309 - val_bbox_output_loss
:put_mae: 0.2133 - class_output_accuracy: 0.6415 - class_output_loss: 1.6532 - loss: 1.9797 - val_bbox_output_loss
:put_mae: 0.2597 - class_output_accuracy: 0.6604 - class_output_loss: 1.6944 - loss: 2.1177 - val_bbox_output_loss
:put_mae: 0.2580 - class_output_accuracy: 0.7075 - class_output_loss: 1.5918 - loss: 2.0317 - val_bbox_output_loss
```

```
# Evaluation of the model based on the training
results_VGG_AUG_LT = model_VGG.evaluate(
    X_test,
    {
        'class_output': y_class_test,
        'bbox_output': y_bbox_test
    }
)

print(results_VGG_AUG)
```

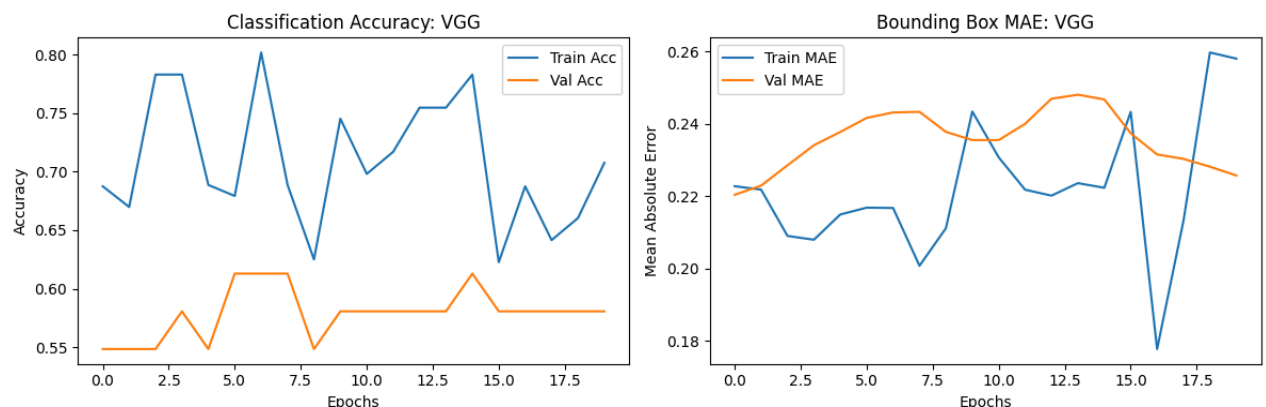
```
1/1 ————— 23s 23s/step - bbox_output_loss: 0.0816 - bbox_output_mae: 0.2257 - class_output_accuar
[1.966860294342041, 1.6085691452026367, 0.07165822386741638, 0.07165822386741638, 0.6129032373428345]
```

```
import matplotlib.pyplot as plt
```

```
# 1. Classification Accuracy Plot
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history_VGG_Aug_LT.history['class_output_accuracy'], label='Train Acc')
plt.plot(history_VGG_Aug_LT.history['val_class_output_accuracy'], label='Val Acc')
plt.title("Classification Accuracy: VGG")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()

# 2. Bounding Box MAE Plot
plt.subplot(1, 2, 2)
plt.plot(history_VGG_Aug_LT.history['bbox_output_mae'], label='Train MAE')
plt.plot(history_VGG_Aug_LT.history['val_bbox_output_mae'], label='Val MAE')
plt.title("Bounding Box MAE: VGG")
plt.xlabel("Epochs")
plt.ylabel("Mean Absolute Error")
plt.legend()

plt.tight_layout()
plt.show()
```



Better accuracy for train and validation data MAE for bounding box prediction not improves with epochs

```
# Prediction for an image
pred_class_VGG_AUG_LT, pred_bbox_VGG_AUG_LT = model_VGG.predict(X_test[:1])
print("Pred_class", pred_class_VGG_AUG_LT)
print("Pred bbox:", pred_bbox_VGG_AUG_LT[0])
print("True bbox:", y_bbox_test[0])
```

```
1/1 ————— 3s 3s/step
Pred_class [[0.03520899 0.01714424 0.03937686 0.69288874 0.00135464 0.02402962
 0.14854491 0.00762098 0.0161425 0.01768856]]
Pred bbox: [0.1781521 0.14925598 0.63462 0.38387227]
True bbox: [0.65789474 0.49561404 0.88888889 0.69736842]
```

```
df_class=df_subset[['CLASS', 'label_encoded']].drop_duplicates().sort_values('label_encoded')
df_class['CLASS'].to_list()
```

```
['articulated_truck',
 'bicycle',
 'bus',
 'car',
 'motorized_vehicle',
 'non-motorized_vehicle',
 'pedestrian',
 'pickup_truck',
 'single_unit_truck',
 'work_van']
```

```
import tensorflow as tf
import numpy as np
import requests
from PIL import Image
from io import BytesIO
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def test_model_on_internet_image(img_url, model, class_names, img_size=224):
    # 1. Download the image
    response = requests.get(img_url)
    img = Image.open(BytesIO(response.content)).convert('RGB')

    # 2. Preprocess (Must match your Training exactly!)
    img_resized = img.resize((img_size, img_size))
    img_array = np.array(img_resized) / 255.0 # Rescale 1./255
    img_array = np.expand_dims(img_array, axis=0) # Add batch dimension

    # 3. Predict
    predictions = model_VGG.predict(img_array)

    # Extract results based on your model's output names
    # Assuming predictions is a list: [class_output_array, bbox_output_array]
    pred_class_logits = predictions[0] # First element is class output
    pred_bbox = predictions[1][0] # Second element is bbox output

    # Get class name
    class_idx = np.argmax(pred_class_logits)
    predicted_label = class_names[class_idx]
    confidence = np.max(tf.nn.softmax(pred_class_logits))

    # 4. Visualize
    fig, ax = plt.subplots(1)
    ax.imshow(img_resized)

    # Draw Bounding Box
    # Convert normalized [0,1] to pixel values
    xmin, ymin, xmax, ymax = pred_bbox
    width = (xmax - xmin) * img_size
    height = (ymax - ymin) * img_size

    rect = patches.Rectangle((xmin * img_size, ymin * img_size),
                             width, height,
                             linewidth=2, edgecolor='r', facecolor='none')
    ax.add_patch(rect)

    plt.title(f"Pred: {predicted_label} ({confidence*100:.2f}%)")
    plt.show()

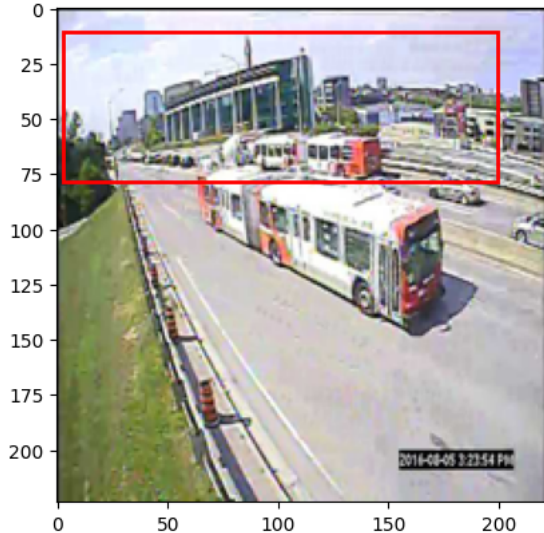
# --- Execution ---
my_classes = ['articulated_truck',
 'bicycle',
 'bus',
```

```
'car',
'motorized_vehicle',
'non-motorized_vehicle',
'pedestrian',
'pickup_truck',
'single_unit_truck',
'work_van']
```

```
url = "https://drive.google.com/uc?export=download&id=14lB-RP8Ip6Foclrl-S6EFhp0yntjks07"
test_model_on_internet_image(url, model_VGG, my_classes)
```

1/1 ————— 1s 852ms/step

Pred: bus (19.62%)



```
# Conversion function of xxyy to the pixel dimension 224*224
```

```
def xxyy_to_pixel(box, img_size=224):
```

```
    x_min, y_min, x_max, y_max = box
```

```
    x_min = int(x_min * img_size)
```

```
    y_min = int(y_min * img_size)
```

```
    x_max = int(x_max * img_size)
```

```
    y_max = int(y_max * img_size)
```

```
    return x_min, y_min, x_max, y_max
```

```
# Visualization Function for prediction output
```

```
import matplotlib.pyplot as plt
```

```
import cv2
```

```
def visualize_prediction(image, pred_box, true_box):
```

```
    img = (image * 255).astype("uint8").copy()
```

```
    # convert boxes
```

```
    pred_xxyy = xxyy_to_pixel(pred_box)
```

```
    true_xxyy = xxyy_to_pixel(true_box)
```

```
    # draw predicted box (GREEN)
```

```
    cv2.rectangle(img,
                  (pred_xxyy[0], pred_xxyy[1]),
                  (pred_xxyy[2], pred_xxyy[3]),
                  (0,255,0), 2)
```

```
    # draw true box (RED)
```

```
    cv2.rectangle(img,
                  (true_xxyy[0], true_xxyy[1]),
                  (true_xxyy[2], true_xxyy[3]),
                  (255,0,0), 2)
```

```
    plt.imshow(img)
```

```
    plt.title("Green = Predicted | Red = Ground Truth")
```

```
    plt.axis('off')
```

```
    plt.show()
```

```
# check prediction for an image data
```